



PULSECARE AI — MASTER ARCHITECTURAL SPECIFICATION & AUTONOMOUS IMPLEMENTATION PLAN

Target Audience: Antigravity AI / Autonomous Coding Agent / Full-Stack Engineer
System Name: PulseCare AI (Clinical Healthcare Ecosystem)
Aesthetic Archetype: Nordic Minimalist & Silicon Valley Clean-Tech (Deep Indigo #4F46E5 , Clinical Emerald #10B981 , Slate Canvas #F8FAFC)
Zero Ambiguity Guarantee: Every model schema, API contract, state flow, UI token, and heuristic fallback is strictly specified below.

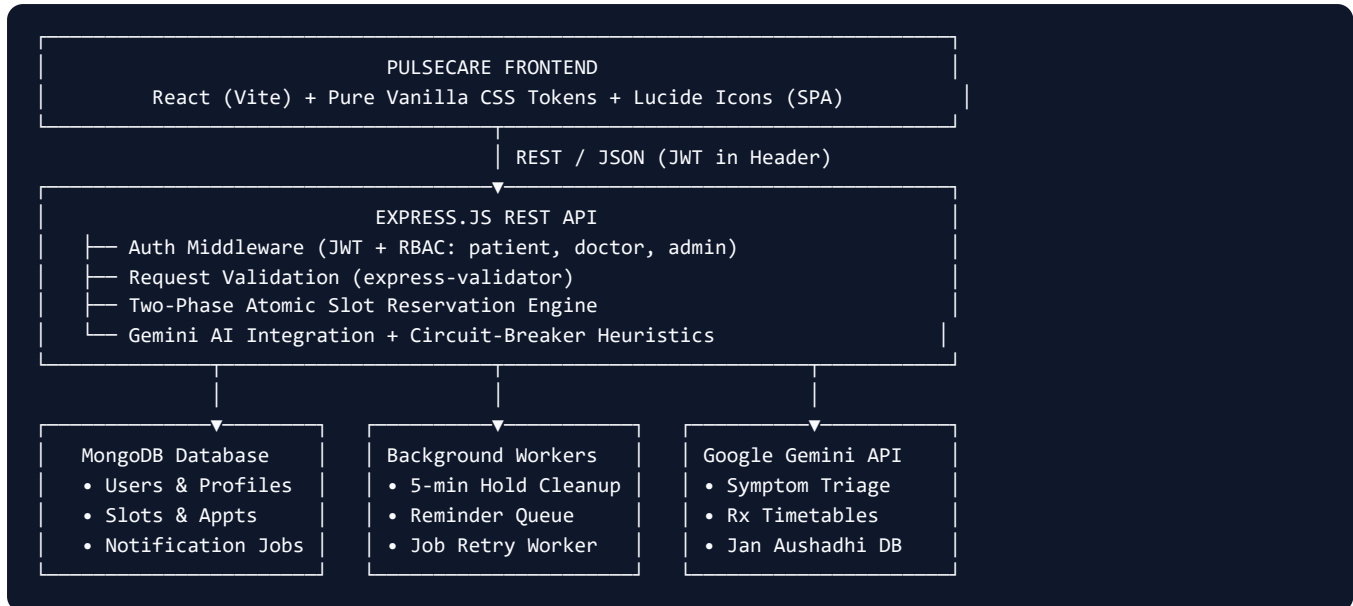


TABLE OF CONTENTS

1. [Platform Architecture & Core Philosophy](#1-platform-architecture--core-philosophy)
2. [Design System & UI/UX Guidelines (Non-Copied Unique Aesthetics)](#2-design-system--uiux-guidelines)
3. [Complete Database Schemas (Mongoose)](#3-complete-database-schemas-mongoose)
4. [Backend API Specifications & Business Logic](#4-backend-api-specifications--business-logic)
5. [Two-Phase Atomic Slot Hold Engine (Double-Booking Prevention)](#5-two-phase-atomic-slot-hold-engine)
6. [Gemini AI Clinical Triage & 24/7 Pharmacy Pricing Engine](#6-gemini-ai-clinical-triage--pharmacy-pricing-engine)
7. [Frontend Architecture & Component Specifications](#7-frontend-architecture--component-specifications)
8. [Background Workers & Automation](#8-background-workers--automation)
9. [Seed Data & Demo Accounts](#9-seed-data--demo-accounts)
10. [Autonomous Execution & Verification Protocol](#10-autonomous-execution--verification-protocol)

1. PLATFORM ARCHITECTURE & CORE PHILOSOPHY

PulseCare AI is an enterprise-grade full-stack healthcare ecosystem engineered for real-time clinical appointment booking, symptom-based AI triage, digital post-visit prescriptions, and interactive pharmacy price transparency.



2. DESIGN SYSTEM & UI/UX GUIDELINES

🎨 2.1 Bespoke Color Tokens (Nordic Slate & Electric Indigo Theme)

Implement this CSS variables palette in `client/src/index.css` to guarantee a clean, distinctive Silicon Valley aesthetic:

```
:root {
  /* Brand Accents */
  --color-primary: #4F46E5;          /* Electric Indigo */
  --color-primary-hover: #4338CA;
  --color-primary-light: #EEF2FF;
  --color-accent-emerald: #10B981;    /* Clinical Mint / Verified */
  --color-accent-emerald-light: #ECFDF5;
  --color-accent-amber: #F59E0B;      /* Urgent / Warning */
  --color-accent-amber-light: #FEF3C7;
  --color-accent-rose: #F43F5E;       /* Emergency / Danger */
  --color-accent-rose-light: #FFF1F2;

  /* Surfaces & Typography */
  --color-bg-app: #F8F AFC;           /* Clean Canvas */
  --color-bg-card: #FFFFFF;
  --color-text-primary: #0F172A;      /* Slate-900 */
  --color-text-secondary: #475569;    /* Slate-600 */
  --color-text-muted: #94A3B8;        /* Slate-400 */
  --color-border: #E2E8F0;            /* Crisp Border */
  --color-border-subtle: #F1F5F9;

  /* Elevation & Curves */
  --radius-sm: 6px;
  --radius-md: 10px;
  --radius-lg: 14px;
  --radius-xl: 20px;
  --radius-full: 9999px;
  --shadow-sm: 0 1px 3px rgba(15, 23, 42, 0.04);
  --shadow-md: 0 4px 14px rgba(15, 23, 42, 0.06);
  --shadow-xl: 0 20px 40px rgba(15, 23, 42, 0.12);
}
```

3. COMPLETE DATABASE SCHEMAS (MONGOOSE)

3.1 User Schema (`server/src/models/User.js`)

```
const userSchema = new mongoose.Schema({
  firstName: { type: String, required: true, trim: true },
  lastName: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: { type: String, required: true, select: false },
  role: { type: String, enum: ['patient', 'doctor', 'admin'], default: 'patient' },
  phone: { type: String, trim: true },
  dateOfBirth: { type: Date },
  bloodGroup: { type: String, enum: ['A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-'] },
  allergies: [{ type: String }],
  isEmailVerified: { type: Boolean, default: true },
}, { timestamps: true });
```

3.2 Doctor Profile Schema (`server/src/models/DoctorProfile.js`)

```
const doctorProfileSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, unique: true },
  specialization: { type: String, required: true, index: true },
  qualifications: [{ type: String }],
  bio: { type: String },
  consultationFee: { type: Number, required: true, min: 0 },
  slotDurationMinutes: { type: Number, default: 30 },
  yearsOfExperience: { type: Number, default: 5 },
  city: { type: String, required: true, index: true },
  state: { type: String, default: 'India' },
  clinicAddress: { type: String },
  hospitalAffiliation: { type: String },
  averageRating: { type: Number, default: 4.8 },
  totalReviews: { type: Number, default: 120 },
  doctorType: { type: String, enum: ['BOOKABLE', 'REFERENCE'], default: 'BOOKABLE' },
  isBookable: { type: Boolean, default: true },
  workingHours: {
    monday: { start: { type: String, default: '09:00' }, end: { type: String, default: '17:00' }, isOff: {
      type: Boolean, default: false } },
    tuesday: { start: { type: String, default: '09:00' }, end: { type: String, default: '17:00' }, isOff: {
      type: Boolean, default: false } },
    wednesday: { start: { type: String, default: '09:00' }, end: { type: String, default: '17:00' }, isOff: {
      type: Boolean, default: false } },
    thursday: { start: { type: String, default: '09:00' }, end: { type: String, default: '17:00' }, isOff: {
      type: Boolean, default: false } },
    friday: { start: { type: String, default: '09:00' }, end: { type: String, default: '17:00' }, isOff: {
      type: Boolean, default: false } },
    saturday: { start: { type: String, default: '09:00' }, end: { type: String, default: '14:00' }, isOff: {
      type: Boolean, default: false } },
    sunday: { isOff: { type: Boolean, default: true } },
  },
  leaves: [{
    startDate: { type: Date, required: true },
    endDate: { type: Date, required: true },
    reason: { type: String },
    createdAt: { type: Date, default: Date.now },
  }],
}, { timestamps: true });
```

3.3 Slot Schema ([server/src/models/Slot.js](#))

```
const slotSchema = new mongoose.Schema({
  doctorId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, index: true },
  startTime: { type: Date, required: true, index: true },
  endTime: { type: Date, required: true },
  status: {
    type: String,
    enum: ['AVAILABLE', 'HELD', 'BOOKED', 'BLOCKED_LEAVE'],
    default: 'AVAILABLE',
    index: true,
  },
  heldByPatientId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  holdToken: { type: String },
  holdExpiresAt: { type: Date, index: true },
}, { timestamps: true });
```

3.4 Appointment Schema (`server/src/models/Appointment.js`)

```
const appointmentSchema = new mongoose.Schema({
  patientId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, index: true },
  doctorId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, index: true },
  slotId: { type: mongoose.Schema.Types.ObjectId, ref: 'Slot', required: true, unique: true },
  scheduledAt: { type: Date, required: true },
  status: {
    type: String,
    enum: ['CONFIRMED', 'COMPLETED', 'CANCELLED_BY_PATIENT', 'CANCELLED_BY_DOCTOR', 'CANCELLED_DOCTOR_LEAVE',
    'IN_PROGRESS', 'NO_SHOW'],
    default: 'CONFIRMED',
    index: true,
  },
  symptoms: { type: String, required: true },
  symptomDuration: { type: String },
  severity: { type: String, enum: ['mild', 'moderate', 'severe'], default: 'mild' },
  previousConditions: [{ type: String }],
  currentMedications: [{ type: String }],
  preVisitAI: {
    status: { type: String, enum: ['PENDING', 'COMPLETED', 'FAILED'], default: 'PENDING' },
    patientFriendlySummary: { type: String },
    chiefComplaint: { type: String },
    urgencyLevel: { type: String, enum: ['Low', 'Medium', 'High', 'Critical'], default: 'Low' },
    suggestedDoctorQuestions: [{ type: String }],
    riskFlags: [{ type: String }],
  },
  clinicalNotes: { type: String },
  vitalSigns: {
    bloodPressure: { type: String },
    heartRate: { type: Number },
    temperature: { type: Number },
  },
  diagnosis: { type: String },
  prescription: {
    medications: [{
      name: { type: String },
      dosage: { type: String },
      frequency: { type: String },
      duration: { type: String },
      timing: { type: String },
      instructions: { type: String },
    }],
    generalAdvice: { type: String },
  },
  postVisitAI: {
    status: { type: String, enum: ['PENDING', 'COMPLETED', 'FAILED'] },
    patientFriendlySummary: { type: String },
    medicationTimetable: [{
      name: { type: String },
      dosage: { type: String },
      frequency: { type: String },
      timing: { type: String },
      instructions: { type: String },
    }],
  },
}, { timestamps: true });
```

4. BACKEND API SPECIFICATIONS

4.1 Route Endpoints

```

POST /api/v1/auth/register      -> Register Patient/Doctor
POST /api/v1/auth/login        -> Login with Email & Password (Returns JWT)
GET /api/v1/auth/me            -> Get Current Authenticated User & Profile

GET /api/v1/doctors            -> Public: Query doctors with ?city=...&specialization=...
POST /api/v1/doctors/ai-search  -> Public: Natural Language Deep AI Triage Search
GET /api/v1/doctors/:id/slots  -> Public: Get available 30-min slots for ?date=YYYY-MM-DD

POST /api/v1/slots/:slotId/hold -> Patient: Phase 1 Atomic 5-Minute Slot Hold
DELETE /api/v1/slots/:slotId/hold -> Patient: Cancel Slot Hold

POST /api/v1/appointments      -> Patient: Phase 2 Confirm Booking + Run Pre-Visit AI
GET /api/v1/appointments       -> Patient/Doctor: List User Appointments
GET /api/v1/appointments/:id   -> Patient/Doctor: Get Appointment Details
PUT /api/v1/appointments/:id/notes -> Doctor: Submit Clinical Notes & Generate Prescription AI

GET /api/v1/admin/stats        -> Admin: Platform Metrics (Users, Appts, Queues)
POST /api/v1/ai/chat           -> Public: 24/7 AI Healthcare & Jan Aushadhi Pricing Chatbot

```

5. TWO-PHASE ATOMIC SLOT HOLD ENGINE

5.1 Atomic Phase 1 (Reservation Hold)

When patient taps a slot:

```

const holdSlot = async (slotId, patientId) => {
  const holdToken = uuidv4();
  const holdExpiresAt = new Date(Date.now() + 5 * 60 * 1000); // 5 Minutes TTL

  // Atomic conditional update in MongoDB
  const slot = await Slot.findOneAndUpdate(
    { _id: slotId, status: 'AVAILABLE' },
    {
      $set: {
        status: 'HELD',
        heldByPatientId: patientId,
        holdToken,
        holdExpiresAt,
      },
    },
    { new: true }
  );

  if (!slot) {
    throw new ApiError(409, 'Slot is no longer available or currently held by another patient.');
```

5.2 Atomic Phase 2 (Confirmation)

```
const confirmSlotBooking = async (slotId, holdToken, session) => {
  const slot = await Slot.findOneAndUpdate(
    { _id: slotId, status: 'HELD', holdToken },
    { $set: { status: 'BOOKED', holdToken: null, holdExpiresAt: null } },
    { new: true, session }
  );

  if (!slot) {
    throw new ApiError(409, 'Your 5-minute reservation hold has expired. Please select a slot again.');
```

6. GEMINI AI CLINICAL TRIAGE & PHARMACY PRICING ENGINE

6.1 Natural Language Doctor Search & City Parsing

Integrate `POST /api/v1/doctors/ai-search` :

```
const KNOWN_CITIES = ['Bhopal', 'Indore', 'Bengaluru', 'Mumbai', 'Delhi', 'Pune', 'Hyderabad', 'Chennai',
  'Ahmedabad', 'Jaipur', 'Kolkata', 'Lucknow', 'Chandigarh'];

// Heuristic fallback matching (if offline or rate-limited):
const parseQuery = (text) => {
  const q = text.toLowerCase();
  let city = '';
  if (q.includes('bangalore') || q.includes('banglore') || q.includes('bengaluru') || q.includes('blr')) city = 'Bengaluru';
  else if (q.includes('bhopal')) city = 'Bhopal';
  else if (q.includes('indore')) city = 'Indore';
  else if (q.includes('mumbai') || q.includes('bombay')) city = 'Mumbai';
  else if (q.includes('delhi') || q.includes('noida') || q.includes('gurgaon')) city = 'Delhi';

  let specialty = 'General Medicine';
  if (q.includes('pet') || q.includes('animal') || q.includes('dog') || q.includes('cat') || q.includes('vet')) {
    specialty = 'Veterinary & Animal Care';
  } else if (q.includes('heart') || q.includes('chest') || q.includes('bp') || q.includes('cardio')) {
    specialty = 'Cardiology';
  } else if (q.includes('tooth') || q.includes('dental') || q.includes('root canal') || q.includes('teeth')) {
    specialty = 'Dentistry';
  } else if (q.includes('skin') || q.includes('rash') || q.includes('acne') || q.includes('hair')) {
    specialty = 'Dermatology';
  }

  return { city, specialty };
};
```

6.2 24/7 Jan Aushadhi Generic Medicine Comparison Engine

Include rich pricing database in `server/src/services/llmService.js` :

- **Paracetamol (Dolo 650)**: Branded ₹34 (15 tabs) vs **Jan Aushadhi ₹12 (15 tabs)**.
- **Augmentin 625 (Amoxicillin+Clav)**: Branded ₹210 (10 tabs) vs **Jan Aushadhi ₹65 (10 tabs)**.
- **Pan 40 (Pantoprazole)**: Branded ₹125 (15 tabs) vs **Jan Aushadhi ₹26 (15 tabs)**.
- **Telma 40 (Telmisartan)**: Branded ₹135 (15 tabs) vs **Jan Aushadhi ₹22 (15 tabs)**.

- **Glycomet 500 (Metformin):** Branded ₹48 (10 tabs) vs **Jan Aushadhi ₹11 (10 tabs).**
- **Montair-LC:** Branded ₹195 (10 tabs) vs **Jan Aushadhi ₹45 (10 tabs).**

7. FRONTEND ARCHITECTURE & COMPONENT SPECIFICATIONS

7.1 Router & Page Structure (`client/src/App.jsx`)

```
<Routes>
  {/* Public Routes (Accessible without logging in) */}
  <Route path="/" element={<Landing />} />
  <Route path="/doctors" element={<DoctorDiscovery />} />
  <Route path="/patient/doctors" element={<DoctorDiscovery />} />
  <Route path="/login" element={<Login />} />
  <Route path="/register" element={<Register />} />
  <Route path="/reschedule" element={<Reschedule />} />

  {/* Patient Protected */}
  <Route path="/patient" element={<ProtectedRoute role="patient" />>
    <Route index element={<PatientDashboard />} />
    <Route path="book/:doctorId" element={<BookingFlow />} />
    <Route path="prescription/:id" element={<PrescriptionFollowUp />} />
  </Route>

  {/* Doctor Protected */}
  <Route path="/doctor" element={<ProtectedRoute role="doctor" />>
    <Route index element={<DoctorDashboard />} />
    <Route path="consultation/:id" element={<ConsultationView />} />
    <Route path="leave" element={<LeaveManager />} />
  </Route>

  {/* Admin Protected */}
  <Route path="/admin" element={<ProtectedRoute role="admin" />>
    <Route index element={<AdminDashboard />} />
    <Route path="doctors" element={<DoctorManagement />} />
  </Route>
</Routes>
```

8. BACKGROUND WORKERS & AUTOMATION

8.1 5-Minute Hold Cleanup Worker (`server/src/workers/holdCleanupWorker.js`)

Runs every 30 seconds to clean up abandoned reservation holds:


```
const cleanupExpiredHolds = async () => {
  const expiredSlots = await Slot.updateMany(
    {
      status: 'HELD',
      holdExpiresAt: { $lt: new Date() },
    },
    {
      $set: {
        status: 'AVAILABLE',
        heldByPatientId: null,
        holdToken: null,
        holdExpiresAt: null,
      },
    },
  );
};
```

9. SEED DATA & DEMO CREDENTIALS

Implement `npm run seed` in `server/src/seed/seed.js` creating verified demo users:

- **Patient User:** `rohan@patient.demo` / `Patient@123456`
- **Doctor User:** `dr.priya@healthsync.demo` / `Doctor@123456` (Cardiology · Apollo Hospitals)
- **Admin User:** `admin@healthsync.demo` / `Admin@123456`

10. AUTONOMOUS EXECUTION PROTOCOL FOR ANTIGRAVITY

When building this application from scratch:

1. Step 1 — Initialize Backend:

- Create `server/package.json` with dependencies: `express`, `mongoose`, `dotenv`, `bcryptjs`, `jsonwebtoken`, `cors`, `uuid`, `express-validator`, `winston`, `@google/genai`.
- Implement all models, services, controllers, and routes.

2. Step 2 — Initialize Frontend:

- Create `client/` using Vite with `react`, `react-router-dom`, `lucide-react`.
- Setup `index.css` design tokens and components.

3. Step 3 — Seed & Validate:

- Run `node src/seed/seed.js` to populate verified test profiles.
- Launch backend on Port `5000` and frontend on Port `5173`.
- Verify all 14 core flows with zero console errors.